COMPSCI 620 Reading Notes

Nikhil Anand

February 9, 2026

Contents

Part I

Reading Notes - Feb 9th

Chapter 1

Understanding Understanding Source Code with Functional Magnetic Resonance Imaging

The paper investigates whether functional magnetic resonance imaging (fMRI) can be used to directly measure what happens in the brain when programmers understand source code. The broader aim is to build a scientific, cognitive basis for program comprehension so that programming tools, languages, and education can be improved.

1.1 Motivation

The main motivation of the paper is that we still don't fully understand what happens cognitively when programmers read and understand code, even though software underpins modern society. Researchers and educators want to improve programming languages, tools, and training, but most existing studies rely on indirect measures—such as performance, surveys, or think-aloud protocols—which are noisy, subjective, and often inconclusive.

But, why do we need a better understanding of program comprehension? Understanding program comprehension is not limited to theory building, but can have real downstream effects in improving education, training, and the design and evaluation of tools and languages for programmers. If direct measures of cognitive effort and difficulty could be obtained and correlated with programming activity, then researchers could identify and quantify which types of activities, segments of code, or kinds of problem solving are troublesome or improved with the introduction of a new language or tool.

The authors argue that to design better tools, languages, and educational approaches, we need direct, objective measurements of the mental processes involved in program comprehension. If researchers could measure cognitive effort and difficulty while someone reads code, they could identify which code structures are hardest to understand, evaluate new tools or languages more rigorously, and better train programmers.

To address this gap, the paper proposes using functional magnetic resonance imaging (fMRI) to observe brain activity during code comprehension. fMRI is widely used in cognitive neuroscience to study complex mental processes, so the authors aim to test whether it is feasible and meaningful to apply it to programming. Their broader goal is to lay the groundwork for a scientific understanding of program comprehension that could eventually answer broader questions, such as:

- What cognitive processes are involved in understanding code?
- How do languages, tools, or training methods affect comprehension?
- Can we predict or improve programming ability?

1.2 Methodology

1.2.1 fMRI

The authors use functional magnetic resonance imaging (fMRI) in a controlled experiment to measure brain activity while participants understand source code. fMRI is a non-invasive brain-imaging technique that measures changes in blood oxygenation, the BOLD (blood oxygen level dependent) signal, to infer neural activity. When a brain region becomes active, it consumes more oxygen, and this change in oxygenated vs. deoxygenated blood can be detected by the scanner. From these signals, researchers can determine which brain areas are active during a task.Because decades of neuroscience research have mapped many brain regions (Brodmann areas) to cognitive functions like language, memory, and attention, observing activation patterns allows researchers to infer which mental processes are involved in program comprehension.

1.2.1.1 Why was fMRI chosen?

The authors chose fMRI because:

- It provides direct physiological measurements of cognitive activity, unlike surveys or performance metrics.
- It allows identification of specific brain regions involved in understanding code.
- It is well established in cognitive neuroscience for studying complex behaviors like language comprehension.
- It can help build a scientific foundation for understanding programming cognition.

1.2.1.2 Challenges

1. fMRI does not measure neural activity directly—it measures the BOLD (blood oxygen level dependent) response, which reflects changes in blood oxygenation when a brain region becomes active. This signal takes a few seconds to appear after a cognitive process begins and peaks around 5 seconds before returning to baseline about 12 seconds after the task ends. Because of this delay, tasks must be carefully timed and long enough (often 30–120 seconds) so that activation builds up and can be measured reliably. Short or poorly timed tasks would produce weak or noisy signals.
2. To obtain statistically reliable results, fMRI studies requires multiple repetitions of similar tasks, averaging brain activity across trials and a careful comparison between experimental and control conditions. This means experiments must use tasks that are consistent in difficulty and duration, which limits how realistic or complex the tasks can be.
3. fMRI is extremely sensitive to movement. Even small movements of the head can introduce artifacts in the signal.

1.2.2 Structure of the Experiment

The study followed standard fMRI experimental design principles:

- Participants lay still in a scanner with their head stabilized to avoid motion artifacts.
- Code snippets were shown on a small mirrored screen.
- Tasks were repeated multiple times to average brain activity.

Each trial included:

1. comprehension task (60s)
2. rest period
3. syntax task (30s)

4. rest period

Rest periods establish a baseline so researchers can compare task-related activation. Participants indicated mentally when they finished a task (using a button) to minimize movement, and correctness was verified afterward.

1.3 Key Findings

1.3.1 Program comprehension activates a specific network of brain regions

The central result is that understanding source code produced a clear and consistent activation pattern in five brain regions, all in the left hemisphere - Brodmann Area 6, 21, 40, 44 and 47. These regions were significantly more active during program comprehension than during syntax-error detection, meaning they are associated specifically with understanding code rather than just reading it or visually scanning it.

1.3.1.1 Working memory and attention play a central role

Two of the activated regions, especially BA 6 and BA 40, are strongly linked to - working memory, divided attention and problem solving. This suggests that when programmers understand code, they must actively:

- Hold variable values and intermediate results in memory
- Track control flow (loops, conditions)
- Coordinate multiple pieces of information simultaneously

In other words, code comprehension heavily relies on cognitive resources similar to those used in reasoning and problem-solving tasks.

1.3.1.2 Strong involvement of language-processing regions

Another major finding is that three activated areas (BA 21, 44, and 47) are typically associated with language processing. This indicates that understanding code engages brain systems used for:

- Processing words and symbols
- Combining them into meaningful structures
- Interpreting semantics

The authors conclude that program comprehension shares similarities with both natural-language and artificial-language processing. This provides empirical support for earlier claims (e.g., Dijkstra) that language skills are important for programming.

1.3.1.3 Evidence for a bottom-up comprehension process

Because the experiment used short code snippets with obfuscated identifiers, it emphasized bottom-up comprehension (understanding code step by step rather than relying on prior knowledge). The activation pattern suggests that programmers:

- Read symbols and words
- Combine them into statements
- Integrate statements into larger semantic chunks
- Keep intermediate values in working memory

This supports a cognitive model in which code comprehension involves incremental integration and memory tracking.

1.3.2 Left-hemisphere dominance

All major activations occurred in the left hemisphere, which is typically associated with analytical reasoning and language. This reinforces the idea that program comprehension relies on structured, language-like cognitive processes.

1.3.3 Feasibility of using fMRI in software-engineering research

Beyond cognitive findings, an important result is methodological: The study demonstrates that fMRI can successfully be used to study program comprehension and produce meaningful activation patterns. This opens the door for future research on programming cognition using neuroscience methods.

1.4 Implications

The results of the paper open several important directions for future research and practical applications.

1.4.1 Study different types of program comprehension

This paper focused mainly on bottom-up comprehension (reading unfamiliar code line by line). A natural next step is to study other forms of comprehension, especially:

- Top-down comprehension: understanding code using prior knowledge, domain familiarity, or meaningful variable names
- Realistic large programs: moving beyond short snippets to more complex codebases
- Use of identifiers and domain knowledge: testing whether meaningful variable names act as "beacons" that change brain activation

Future experiments could compare obfuscated vs. meaningful code to see how memory and recognition affect comprehension.

1.4.2 Study code writing, not just code reading

This study only examined understanding code. Another major step is to study code generation or editing, which may involve different cognitive processes such as creativity and synthesis. Possible experiments include

- Compare brain activity during coding vs. reading
- Examine whether writing code activates right-hemisphere or creative regions
- Study subvocal speech or internal narration while coding

This would help build a more complete model of programming cognition.

1.4.3 Compare novice and expert programmers

A key open question is how expert programmers differ from average programmers. Future work could:

- Compare activation patterns of novices vs. experts
- Study whether experts show more efficient or reduced brain activation
- Identify cognitive traits linked to programming skill

Such research might help predict programming aptitude or guide training methods.

1.4.4 Evaluate programming languages and tools

The methodology could be used to evaluate whether certain language features or tools make code easier to understand. Possible experiments:

- Compare object-oriented vs. functional code
- Test different naming conventions or syntax styles
- Evaluate IDE tools or visualization systems
- Measure cognitive load across language designs

fMRI could help determine whether a language feature actually reduces mental effort rather than relying on subjective reports.

1.4.5 Compare code comprehension with natural language

Because the study found strong language-related brain activation, future work could compare:

- Code comprehension vs. reading natural language passages
- Programming languages vs. artificial grammars
- Multilingual programmers vs. monolingual programmers

This could clarify whether programming is more like language comprehension or mathematical reasoning.

1.4.6 Overall implication

The most important implication is that this paper establishes a new research direction: using neuroscience tools to study programming. The next steps involve scaling up the realism of tasks, comparing different kinds of programmers and programming activities, and using brain-based measurements to inform education, language design, and tool development.